Project Proposal Template

Real-Time Supply Chain Monitoring Platform

Project

Real-Time Supply Chain Monitoring Platform

Objective

To build a data platform that collects supply-chain data, validates and processes it, stores reliable records, and displays useful monitoring indicators such as shipment status, delivery delays, and inventory changes.

The project will start with a working batch pipeline and then extend to Big Data processing and a simulated streaming prototype.

Description

The team will use a suitable public or sample supply-chain dataset. Python will ingest the data, Pandas will support initial cleaning and validation, and Microsoft SQL Server will store structured records for querying.

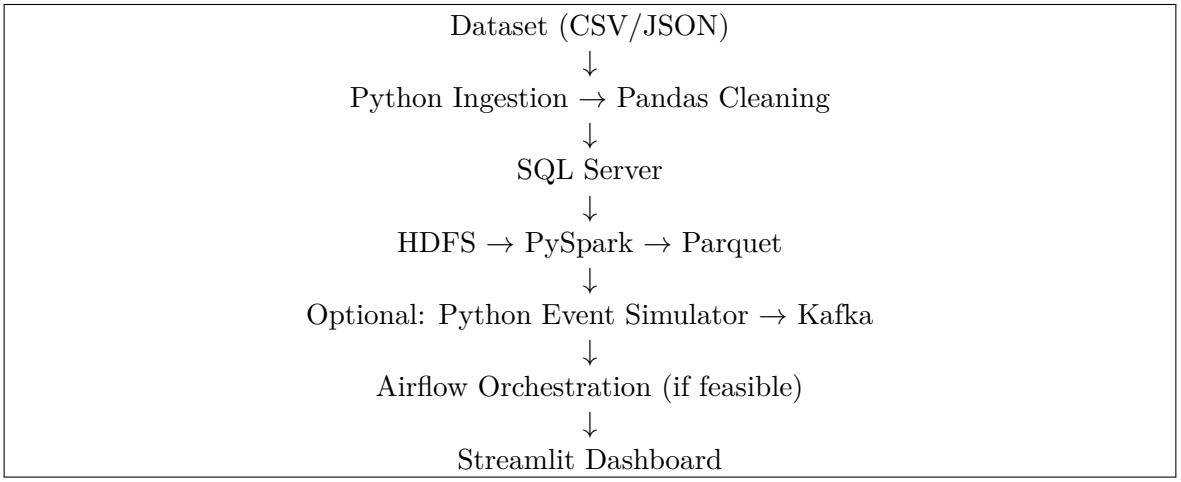
The Big Data phase will introduce HDFS for distributed storage, PySpark for distributed transformations, and Parquet for curated outputs.

If time and setup allow, Kafka will demonstrate event streaming. If no live source is available, a Python script will replay dataset records as simulated events. Airflow may orchestrate pipeline tasks, and Streamlit will display selected monitoring indicators through a dashboard.

The team will prioritize a working and testable minimum viable project (MVP). Simulated events will be clearly labelled and will not be presented as live operational data.

Tools & Design

Proposed data flow



- **Python and Pandas:** Data ingestion, profiling, cleaning, and validation.
- **CSV/JSON or public dataset:** Initial data source.
- **SQL Server and SQL:** Structured storage, queries, and monitoring metrics.
- **SSMS:** Client application used to manage and query SQL Server.
- **HDFS, PySpark, and Parquet:** Distributed storage, processing, and analytical outputs.
- **Streamlit:** Dashboard for selected supply-chain indicators.
- **Git and GitHub:** Version control and team collaboration.
- **Optional extensions:** Kafka for simulated event streaming and Airflow for workflow scheduling and monitoring.

Team Members & Responsibilities

All six members will contribute to implementation, review, integration, and testing. The responsibilities below are starting points, not isolated assignments. Every member should complete practical work and review or test another member's contribution.

Team Member	Responsibilities
Yasmin Kasem Mohammed Sabry	Schema and workflow design; Python/Pandas contribution; SQL validation; Spark-output review; dashboard checks; integration and documentation.
Nourhan Mohamed Ahmed Sultan	Dataset and source exploration; validation rules; SQL tables and queries; test cases; dashboard checks and documentation.
Abdelrahman Samir Mohamed	Python ingestion; transformation logic; SQL integration; PySpark contribution; optional Kafka prototype; pipeline debugging and testing.
Nada Alaa El-Din Ibrahim	Data dictionary and schema; cleaning rules; SQL analysis; Spark-output checks; dashboard metrics and validation.
Salama Wael Salama Ahmed	Source-data profiling; data-quality checks; edge-case tests; integration checks; setup guide and presentation.
Hanan Hosny Mesaad El-Adl	Sample-data preparation; transformation checks; SQL queries; optional streaming tests; dashboard documentation and end-to-end validation.

All members will participate in code reviews and testing. Tasks can be completed in pairs, and responsibilities may be adjusted according to progress and availability.

Week 1: Scope, Dataset, and Design

- Agree on the MVP, monitoring questions, and suitable dataset. All members inspect assigned fields or records and identify data-quality issues.
- Draft the schema, data dictionary, metrics, and initial validation rules. Yasmin and Nada coordinate the draft; everyone reviews it.
- Set up the shared GitHub repository, folder structure, and collaboration workflow. Abdelrahman and Hanan prepare the initial structure.
- Draft the Python ingestion approach and SQL Server table design. Nourhan and Salama prepare sample queries and test cases.
- Check tool availability on team machines and document setup risks.

Deliverables

- Agreed project scope and monitoring questions.
- Selected dataset and source documentation.
- Initial data profiling and data dictionary.
- Draft schema and validation rules.
- Shared repository and initial architecture diagram.

Week 2: Ingestion, Cleaning, and Storage

- Build the Python ingestion and Pandas cleaning pipeline in small, reviewable parts.
- Create SQL Server tables and load a sample dataset. Members work in pairs on Python and SQL tasks.
- Check missing values, duplicates, data types, invalid records, and row counts.
- Document how invalid records are handled and test whether the pipeline can be run repeatedly.
- Begin HDFS and PySpark setup after the basic batch pipeline is working.

Deliverables

- Repeatable Python/Pandas ingestion pipeline.
- Initial cleaning and validation rules.
- SQL Server tables populated with the dataset.
- Initial SQL queries and data-quality results.
- HDFS/PySpark setup notes.

Week 3: Big Data, Queries, and Streaming

- Use PySpark to transform staged data and write curated outputs in Parquet format.
- Compare schemas, row counts, and selected quality checks between batch and Spark outputs.
- Write SQL queries for supported indicators, such as shipment status, delivery delays, or inventory levels.
- If the core pipeline is stable, build a Kafka producer/consumer demonstration.
- If no live feed exists, replay records using Python and label the events as simulated.
- Integrate components in pairs and test work developed by another teammate.

Deliverables

- PySpark transformation workflow.
- Curated Parquet outputs.
- Recorded comparison and validation results.
- SQL queries for selected monitoring indicators.
- Optional Kafka prototype or documented reason for deferring it.

Week 4: Dashboard, Testing, and Presentation

- Build and integrate the Streamlit dashboard using metrics traceable to SQL queries or curated data.
- If feasible, configure Airflow to schedule and monitor the agreed pipeline tasks.
- Run end-to-end tests from source data through processing and storage to dashboard output.
- Check edge cases and confirm that dashboard totals match independently verified query results.
- Each member tests a component developed by another teammate.
- Prepare setup instructions, screenshots, architecture documentation, and presentation slides.

Deliverables

- Integrated core pipeline and Streamlit dashboard.
- End-to-end test cases and recorded results.
- Airflow workflow, if feasible.
- README and setup instructions.
- Architecture diagram and known limitations.
- Final presentation.

Expected Project Deliverables

- Project scope and architecture diagram.
- Dataset documentation and data dictionary.
- Python ingestion, cleaning, and validation pipeline.
- SQL Server tables and analytical queries.
- HDFS/PySpark workflow and Parquet outputs, subject to successful setup.
- Streamlit monitoring dashboard.
- Optional Kafka prototype and Airflow orchestration.
- Test results, setup guide, and final presentation.